

Results and Evaluation

System Architecture Evaluation

Comparison: Monolithic MVC vs. Hexagonal Architecture

This section evaluates this system's architecture (**Hexagonal Architecture**) against the monolithic **MVC** architecture like the one used in **Moodle**.

Feature	MVC	Hexagonal Architecture
Coupling	The model is tightly coupled to the view and controller.	The core domain is decoupled from the dependencies of the other system layers.
Testability	Hard to test due to the tight coupling between layers. If a function that requires a database connection is tested, it must have a database connection.	Easy to test due to the separation of concerns and layers. A mock database connection can be used to test the same function without a database connection.
Replaceability	Changing the database or web framework requires a system-wide rewrite.	Changing the database or web framework is easy and requires rewriting only the adapters without touching the core domain of the system.
Complexity	Lower initial complexity	Higher initial complexity
Framework lock-in	The framework is tightly coupled to the system, making it difficult to switch to a different framework.	The framework is decoupled from the core domain of the system, and only located in the adapters layer

The hexagonal architecture approach seems at first glance to be more complex and more verbose than the traditional **MVC**. But, it provides a better architectural solution when the codebase grows, and the system becomes more *complex*.

Performance Evaluation

The system performance was evaluated using:

- **Grafana K6**: A load testing tool for **APIs**, **microservices**, and **websites**.

The hardware used for the evaluation is:

- CPU: AMD Ryzen 7 4800H (16 Threads) @ 4.30GHz
- RAM: 16GB
- GPU: NVIDIA RTX 3060
- OS: Arch Linux 64-bit (Kernel 7.0.0-1-cachyos)
- Docker Engine: 29.4.0

The **k6** script is as follows:

```
import http from 'k6/http';
import { check, sleep } from 'k6';

export const options = {
  vus: 100,
  duration: '30s',
};
```

```

export default function () {
  const res = http.get('https://localhost:8443/health', {
    tlsInsecureSkipVerify: true,
  });
  check(res, {
    'status is 200': (r) => r.status === 200,
  });
  sleep(1);
}

```

Response Time

A different request endpoint was tested and aggregated into a **CSV** file, then visualized.

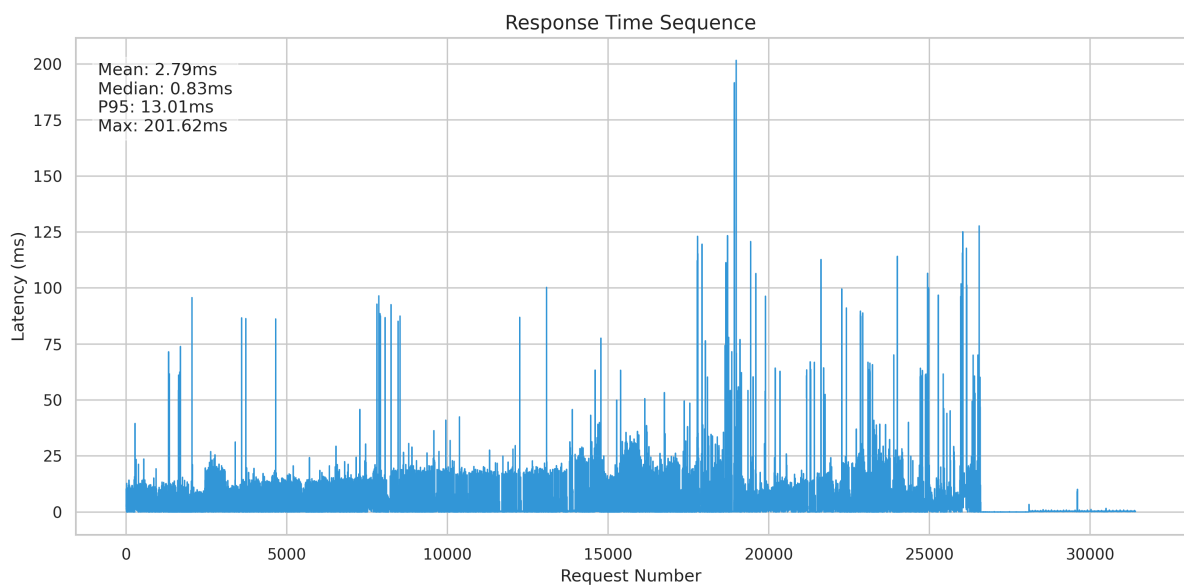


Figure 1: Response time of the system under load using **k6** and **Matplotlib**

Memory Usage

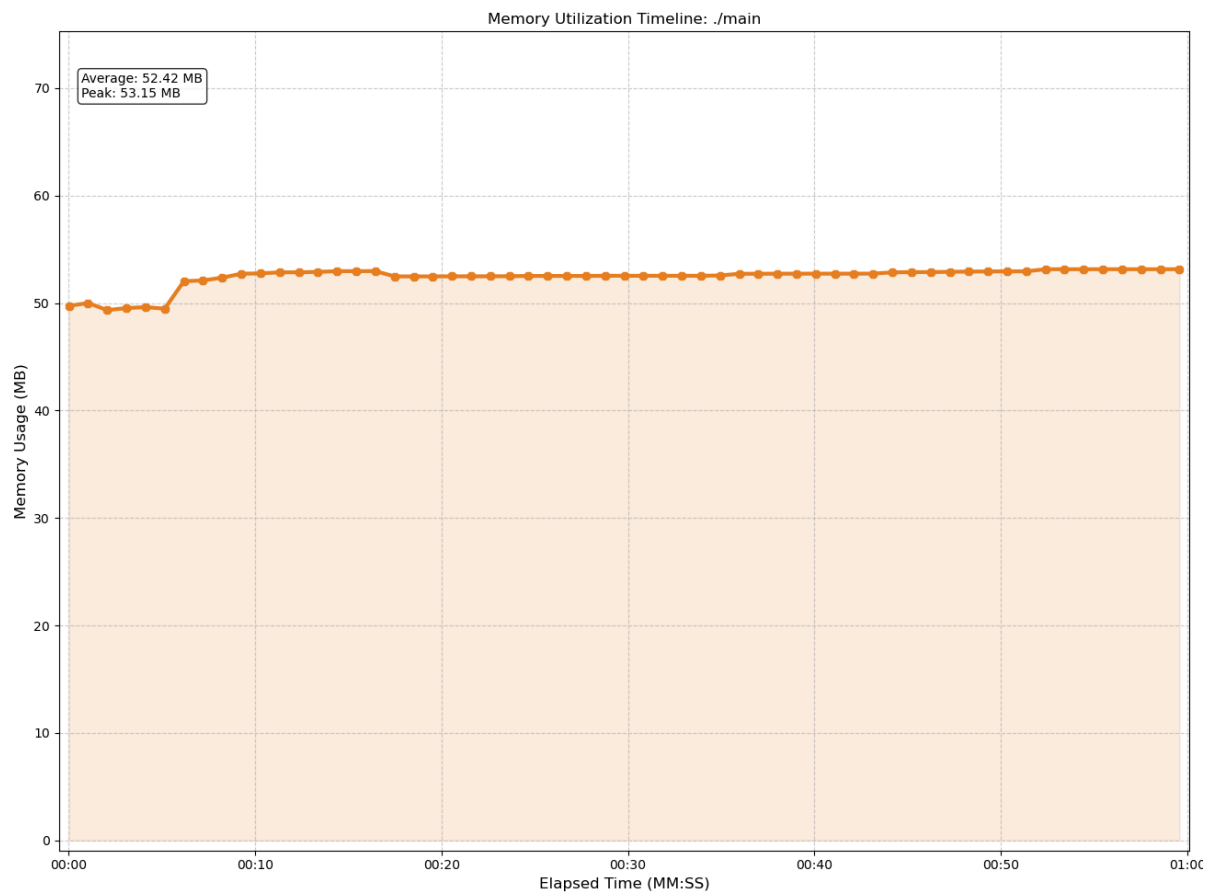


Figure 2: Memory usage of the system under load using **k6**

Comparison with Moodle, and MCQUOT

Metric	Moodle	MCQUOT	This System
Response Time (Average)	700ms	660ms	3ms
Memory Usage (Idle)	97MB	83MB	35MB
Memory Usage (Peak)	1GB to every 15 users	1GB to every 20 users	52MB to every 100 users
Startup time	1-2s	1-2s	<100 ms
Language	PHP	PHP	Go
Concurrency model	Multi-process	Multi-process	Goroutines (lightweight; M:N scheduling)
Type safety	Dynamic typing	Dynamic typing	Strong static typing
Binary size	N/A	N/A	60MB (self-contained binary)
Source code size	434.8 MB	422.9 MB	58.4 MB

Solved Problems of the Previous MCQUOT System

The previous system was suffering from a lot of *architectural*, *performance*, *security*, and *functional* problems. Some of these functional problems are:

Database question duplication

The previous system had no mechanism to prevent *duplication* of questions in a single exam context. So the same question could appear multiple times in the same exam.

To prevent this, we added a constraint that computes the MD5 hash of the question text and the exam uuid to ensure that there is no duplication of questions in the same exam.

Database question randomization

The previous system had no mechanism to *randomize* the questions. To *randomize* the questions in a single exam attempt, we added a hash of the exam's uuid with the exam_attempt's uuid to generate a random seed for the random number generator to order the questions on it.

Question limitation

The previous system had only one type of question, and it was simply the text question type. This system supports multiple types of questions, such as:

Text questions

The screenshot shows a question interface for a text question. At the top, it says "Q.5. Which of the following is NOT a property of a standard rotation matrix R ?" with a "20 Marks" indicator. Below the question, the rotation matrix R is defined as
$$R = \begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix}$$
. There are four options: A, B, C, and D. Option D is highlighted with a green border, indicating it is the correct answer. At the bottom right, there are "Edit" and "Delete" buttons.

Q.5. Which of the following is NOT a property of a standard rotation matrix R ? 20 Marks

$$R = \begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix}$$

A The determinant of the matrix is always equal to 1 ($\det(R) = 1$)

B The matrix is orthogonal, meaning $R^T = R^{-1}$.

C Rotation matrices preserve the length of vectors (isometry).

D The eigenvalues of a 2D rotation matrix for any $\theta \neq 0, \pi$ are always real numbers.

Edit Delete

Figure 3: Text question type

Code highlighted questions

Q.1. What the following code is using ? 1 Marks

```
fn main() {
    use std::collections::HashMap;

    let mut scores = HashMap::new();
    scores.insert(String::from("Blue"), 10);

    scores.entry(String::from("Yellow")).or_insert(50);
    scores.entry(String::from("Blue")).or_insert(50);

    println!("{:?}", scores);
}
```

☐ A Structs

☒ B Hashmaps

☐ C Vec3

☐ D Classes

[Edit](#) [Delete](#)

Figure 4: Code highlighted question type

Image questions

Q.3. Which of the following sequences of input values represents a valid path that starts at *Start* and ends at an q^F state? 1 Marks

```

graph LR
    start((start)) --> q0((q0))
    q0 -- a --> q10((q10))
    q0 -- b --> q01((q01))
    q10 -- a --> q20((q20))
    q10 -- b --> q11((q11))
    q01 -- a --> q11
    q01 -- b --> q02((q02))
    q20 -- a --> q20
    q20 -- b --> q21((q21))
    q11 -- a --> q21
    q11 -- b --> q12((q12))
    q02 -- a --> q12
    q02 -- b --> q02
    q12 -- a --> qF(((qF)))
    q12 -- b --> q12
    qF -- a,b --> qF
    
```

☐ A $a \rightarrow b \rightarrow a \rightarrow a$

☐ B $b \rightarrow b \rightarrow a \rightarrow a$

☒ C $a \rightarrow a \rightarrow b \rightarrow b$

☐ D $b \rightarrow a \rightarrow b \rightarrow a$

[Edit](#) [Delete](#)

Figure 5: Image question type

Manual Data Entry

The previous system had no mechanism to automate the data entry process. This system is solving this problem by implementing a **CSV *import/export*** feature to add questions, exams, users, and subjects to the system.